

Smart Speed Monitoring System for Road Safety

Task 2 – System Specification

Prepared by

Ayush Kishor Dharmik

Arnita Infotech – Embedded Systems Internship

Submission Date: 10 July 2026

Table of Contents

Section	Page No.
1.0 Introduction	3
2.0 System Overview	3
3.0 System Architecture	3
4.0 Hardware Specification	5
5.0 Software Specification	5
6.0 Operating Voltage	6
7.0 Response Time	7
8.0 Memory Requirements	8
9.0 Communication Protocol	9
10.0 Arduino Pin Configuration	11
11.0 System Workflow	11
12.0 Technical Specification Summary	12
13.0 Design Decisions	13
14.0 Conclusion	14

1.0 Introduction

This document constitutes Task 2 of the Embedded Systems Internship undertaken at Arnita Infotech and presents the System Specification for the Smart Speed Monitoring System for Road Safety. Where Task 1 established the requirement analysis for the project, this document translates those requirements into a concrete technical specification, defining the exact hardware, firmware, electrical, and communication parameters that govern the design.

The purpose of this System Specification is to provide a single authoritative reference describing how the Arduino Uno, the paired IR sensor modules, the I²C OLED display, and the active buzzer are configured and interconnected, and to record the electrical, timing, memory, and communication characteristics that the firmware must satisfy. This document is intended to guide the subsequent circuit assembly, firmware implementation, and testing phases, and to serve as a technical record that can be reviewed independently of the requirement-analysis document.

2.0 System Overview

The Smart Speed Monitoring System operates on a dual-point, break-beam timing principle. Two IR sensor modules are mounted at a fixed, known distance apart along a vehicle's path of travel. As a vehicle passes the first sensor, its digital output changes state, and the Arduino Uno begins an internal timer using the microcontroller's `millis()` function. As the same vehicle subsequently passes the second sensor, its output changes state, stopping the timer and yielding the elapsed transit time between the two fixed points.

Using the known sensor separation and the measured elapsed time, the firmware computes the vehicle's speed via the standard relationship $\text{speed} = \text{distance} / \text{time}$, converting the result into kilometres per hour for display. If the computed value exceeds a threshold defined in firmware, the Arduino Uno drives the active buzzer to produce an audible over-speed alert. Once the cycle completes, the system automatically resets its internal state and returns to a ready condition, prepared to time the next passing vehicle without any manual intervention.

3.0 System Architecture

The system architecture reflects a straightforward, single-microcontroller design in which the Arduino Uno acts as the central processing element for all sensing, computation, and output-driving tasks. Figure 1 illustrates the architecture as a signal path, beginning at the vehicle and

IR Sensor 1, through timer start and stop events triggered by the two sensors, into the speed-calculation routine executed on the Arduino Uno, and finally out to the OLED display and buzzer.



Figure 1: System Architecture / Block Diagram

4.0 Hardware Specification

The table below specifies each hardware component used in the prototype, including quantity, key technical specification, and its functional role within the system.

Component	Quantity	Specification	Purpose
Arduino Uno R3	1	ATmega328P, 16 MHz, 5V logic	Central controller; timing, speed calculation, and output control
IR Sensor Module	2	Digital output, adjustable detection range	Marks vehicle entry and exit for timing
OLED Display	1	0.96" 128×64 pixels, I ² C, SSD1306 controller	Displays computed speed and system status
Active Buzzer	1	5V DC, fixed-tone	Audible over-speed alert
Breadboard	1	Standard solderless prototyping board	Interconnects all components
Jumper Wires	As required	Male-to-male / male-to-female	Electrical connections between components
USB Cable	1	USB-A to USB-B, 5V	Power supply and firmware upload

5.0 Software Specification

The table below lists the software tools and libraries used to develop, simulate, and version-control the firmware for this project.

Software	Purpose	Version
Arduino IDE	Firmware development, compilation, and upload to the Arduino Uno	2.x
Embedded C/C++	Implementation language for sensor, timing, and control logic	N/A
Wire Library	I ² C communication between Arduino Uno and OLED display	Bundled with IDE
Adafruit SSD1306 Library	Low-level driver for the SSD1306 OLED controller	Latest stable
Adafruit GFX Library	Graphics primitives (text, shapes) used to render the display	Latest stable
GitHub	Source-code version control and change history	N/A
Wokwi Simulator	Circuit and firmware simulation prior to hardware assembly	N/A

6.0 Operating Voltage

The Arduino Uno R3 operates at a logic level of 5V DC, supplied in this project through the onboard USB connector rather than the barrel-jack input, since a USB connection to a host computer or a standard 5V USB adapter is sufficient for both programming and continuous operation during testing.

All peripheral components in this design, the IR sensor modules, the OLED display, and the active buzzer, are compatible with the 5V logic rail produced by the Arduino Uno. The 0.96-inch I²C OLED module includes an onboard voltage regulator that accepts a 3.3V–5V supply and internally manages the drive voltage of the SSD1306 controller, allowing it to be connected directly to the Arduino's 5V and GND pins without any additional level-shifting circuitry. This single-voltage-domain design was a deliberate simplification: it removes the need for separate regulators or level shifters and reduces the component count and potential points of failure in the prototype.

The specification table below summarises the relevant voltage parameters for each component in the system.

Parameter	Value
Operating Voltage (Arduino Uno logic)	5V DC
Recommended Input Voltage (via USB, as used)	5V DC
Recommended Input Voltage (via barrel jack, not used)	7–12V DC
IR Sensor Module Operating Voltage	5V DC
OLED Display Operating Voltage	3.3V–5V DC (onboard regulation)
Active Buzzer Operating Voltage	5V DC

7.0 Response Time

The overall responsiveness of the Smart Speed Monitoring System depends on four sequential timing contributions, each of which has been considered in the firmware design.

Vehicle Detection Time: The IR sensor modules used in this project respond to a change in reflected or interrupted infrared light within microseconds, and the Arduino Uno's digital input read is effectively instantaneous relative to vehicle speeds encountered in the test setup. Detection latency at each sensor is therefore negligible compared to the vehicle's transit time between the two sensors.

Speed Calculation Time: The speed computation performed by the firmware consists of a single subtraction of two `millis()` timestamps followed by a division and a unit-conversion multiplication. On the ATmega328P running at 16 MHz, this arithmetic completes in well under a millisecond, making the calculation time effectively instantaneous relative to the other stages.

OLED Display Update Time: Refreshing the SSD1306-based OLED over I²C involves transmitting a new frame buffer to the display controller. In practice, a partial-content update

of this kind, showing an updated numeric speed value, completes within approximately 20–40 milliseconds using the Adafruit SSD1306 library at standard I²C clock speeds, which is fast enough to appear immediate to a human observer.

Buzzer Activation Time: Driving the active buzzer is a single `digitalWrite()` instruction and takes effect within microseconds; the buzzer's audible response is limited only by its own internal electromechanical or piezoelectric response time, which is on the order of a few milliseconds.

Taken together, the expected overall system response time, from the moment a vehicle clears the second IR sensor to the moment the updated speed is visible on the display and, where applicable, the buzzer sounds, is expected to remain within approximately 50–150 milliseconds, a delay that is imperceptible as a delay to a driver or observer at the roadside.

8.0 Memory Requirements

The Arduino Uno R3 is built around the ATmega328P microcontroller, which provides 32 KB of flash memory (of which approximately 0.5 KB is reserved for the bootloader), 2 KB of SRAM, and 1 KB of EEPROM. The table below summarises these resources alongside their intended use in this project.

This memory budget is sufficient for the current design. The compiled firmware, including the Wire library for I²C communication and the Adafruit_GFX and Adafruit_SSD1306 display libraries, is expected to occupy a modest fraction of the available 32 KB of flash. The most significant consumer of SRAM is the SSD1306 driver's internal display buffer, which reserves 1 KB to hold the full 128×64 monochrome frame before it is transmitted over I²C; combined with the firmware's timing variables, sensor-state flags, and the Arduino runtime overhead, total SRAM usage remains comfortably within the 2 KB available. EEPROM is not used by the current firmware but is reserved as a candidate location for storing a configurable speed threshold or sensor-spacing calibration value in a future revision.

Memory Type	Total Capacity	Use in This Project
Flash Memory	32 KB (≈ 0.5 KB used by bootloader)	Stores the compiled firmware, including the Wire, Adafruit_GFX, and Adafruit_SSD1306 libraries
SRAM	2 KB	Holds the 1 KB SSD1306 display buffer plus timing variables, sensor-state flags, and runtime overhead
EEPROM	1 KB	Not used in the current firmware; reserved for future storage of a configurable speed threshold or sensor spacing

9.0 Communication Protocol

This project relies exclusively on the Inter-Integrated Circuit (I²C) protocol to communicate between the Arduino Uno and the OLED display, and this section explains the protocol as it applies specifically to that link.

I²C is a synchronous, two-wire serial communication protocol that uses a shared data line (SDA) and a shared clock line (SCL) to allow a master device, the Arduino Uno in this design, to exchange data with one or more addressable slave devices. Each slave device responds only to communications addressed to its unique bus address, allowing multiple peripherals to share the same two physical wires.

The OLED display used in this project uses I²C because the SSD1306 controller embedded in the module is designed to operate over this interface, and because I²C requires only two signal lines in addition to power and ground, which keeps the wiring between the Arduino Uno and the display minimal, a significant advantage on a breadboard prototype where wiring complexity directly affects reliability.

On the Arduino Uno, the I²C bus is fixed to two dedicated analog pins: A4 serves as SDA, the bidirectional data line over which addresses and data bytes are transmitted, and A5 serves as SCL, the clock line generated by the Arduino as I²C master to synchronise data transfer with the display.

The principal advantages of I²C in this project are its minimal pin count, its native support in the Arduino Wire library, and the fact that the OLED module's SSD1306 controller is purpose-built for this interface, which together eliminate the need for any additional interface hardware.

For context, the table below briefly compares I²C with two other common embedded communication protocols, UART and SPI, as well as the automotive-grade CAN bus. None of these protocols is used in the current design, and they are included here only to clarify why I²C was the appropriate choice for this project.

SDA and SCL Pins on the Arduino Uno

Signal	Arduino Uno Pin	Function
SDA	A4	Bidirectional serial data line carrying addresses and data between the Arduino Uno and the OLED display
SCL	A5	Serial clock line generated by the Arduino Uno (I ² C master) to synchronise data transfer

Protocol Comparison (Context Only)

Protocol	Brief Description	Used in This Project
UART	Asynchronous, point-to-point serial link commonly used for serial monitor / debug communication	No
SPI	Synchronous, high-speed serial bus using separate MOSI, MISO, and clock lines with a chip-select per device	No
CAN	Robust, multi-master automotive/industrial bus designed for noisy, multi-node environments	No
I ² C	Synchronous, two-wire (SDA/SCL) bus supporting multiple addressable devices on a shared line	Yes

10.0 Arduino Pin Configuration

The table below documents the complete pin assignment used to connect the IR sensors, the active buzzer, and the OLED display to the Arduino Uno.

Arduino Pin	Connected Device	Function
D2	IR Sensor 1	Vehicle Detection (Timer Start)
D3	IR Sensor 2	Vehicle Detection (Timer Stop)
D8	Active Buzzer	Over-Speed Alert
A4	OLED SDA	I ² C Data
A5	OLED SCL	I ² C Clock

11.0 System Workflow

Figure 2 presents the complete firmware workflow as a flowchart, covering the system's behaviour from power-on through vehicle detection, speed calculation, display update, and the conditional over-speed alert, before returning to a ready state for the next vehicle.

The workflow is structured as a continuous loop: after the system completes a measurement cycle, whether or not the over-speed threshold was exceeded, control returns to the waiting state so that the device can process the next vehicle without requiring a manual reset. This single decision point, comparing the computed speed against the configured limit, is the only branch in the control flow, which keeps the firmware logic straightforward to implement, test, and verify.

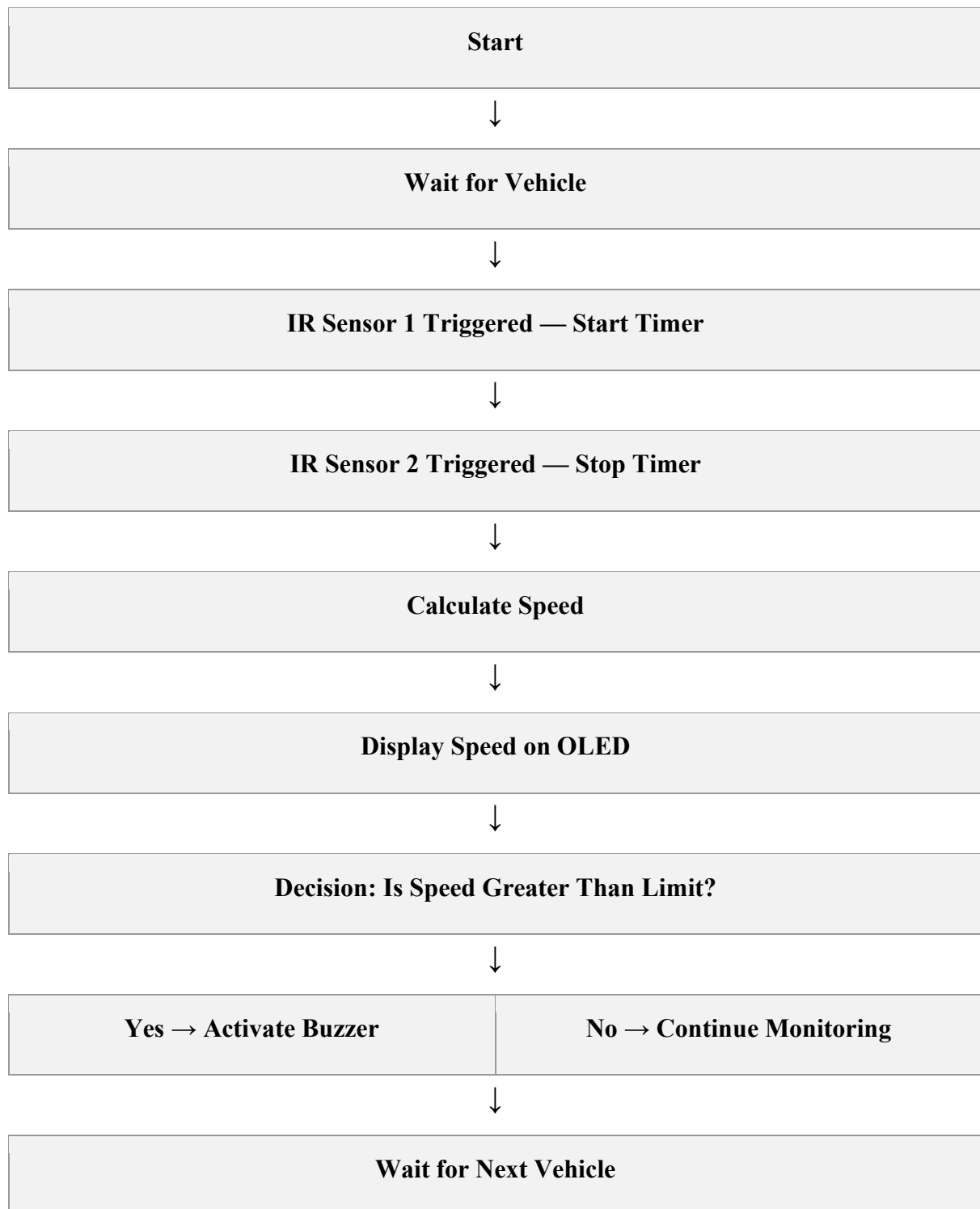


Figure 2: System Workflow Flowchart

12.0 Technical Specification Summary

The table below consolidates the key technical parameters of the Smart Speed Monitoring System into a single reference summary.

Parameter	Specification
Microcontroller	ATmega328P (Arduino Uno R3)
Clock Frequency	16 MHz
Operating Voltage	5V DC
Input Voltage (as used)	5V DC via USB
Display Resolution	128 × 64 pixels (0.96" OLED)
Sensor Type	Digital-output IR sensor module × 2
Detection Method	Dual-point break-beam timing over a fixed known distance
Communication Protocol	I ² C (SDA: A4, SCL: A5)
Programming Language	Embedded C/C++ (Arduino framework)
Power Source	USB, 5V DC
Response Time	Approximately 50–150 ms end-to-end
Estimated Memory Usage	≈1.2 KB of 2 KB SRAM; well within 32 KB Flash

13.0 Design Decisions

Arduino Uno: The Arduino Uno R3 was selected as the central controller because its ATmega328P microcontroller provides sufficient processing speed and I/O for this project's single-lane, single-vehicle timing task, while its widespread availability, extensive documentation, and low cost make it well suited to an internship-scale prototype that must be built, debugged, and demonstrated within a limited timeframe.

IR Sensors: A pair of digital-output IR sensor modules was chosen over ultrasonic or camera-based alternatives because they provide a fast, unambiguous binary trigger signal at the moment a vehicle passes, which is exactly the event needed to start and stop the timing

measurement, and because they are inexpensive and simple to interface directly to the Arduino Uno's digital input pins without additional signal conditioning.

OLED Display: A 128×64 I²C OLED display was selected over an LCD alternative because it offers higher contrast and sharper text rendering at a comparable cost, consumes less power, and, critically, requires only two signal wires via I²C rather than the six or more parallel data lines typically needed for a character LCD, simplifying the wiring on the prototype breadboard.

Active Buzzer: An active buzzer was chosen over a passive buzzer because it contains its own internal oscillator circuit and produces a fixed-tone audible alert from a simple digital HIGH signal, removing the need for the firmware to generate a tone waveform in software and keeping the over-speed alert logic to a single `digitalWrite()` call.

I²C Communication: I²C was the natural communication choice for this project because the OLED module's SSD1306 controller is built specifically for this interface, and because the two-wire bus keeps the wiring count low, which was a priority given that the Arduino Uno's digital pins were already committed to the two IR sensors and the buzzer.

USB Power Supply: Powering the system through the Arduino's USB connector was chosen over a separate barrel-jack power supply because it allows the same cable to serve both firmware programming and continuous operation during testing, and because the entire system's current draw, dominated by the Arduino board, the OLED display, and the buzzer, remains comfortably within the current a standard USB port or USB power adapter can supply.

14.0 Conclusion

This System Specification has defined the precise hardware configuration, electrical parameters, memory budget, communication protocol, pin assignments, and firmware workflow that govern the Smart Speed Monitoring System for Road Safety. Building on the requirement analysis established in Task 1, this document fixes the concrete technical decisions, an Arduino Uno operating at 5V, two IR sensors for dual-point timing, an I²C-driven 128×64 OLED display, and an active buzzer for over-speed alerting, that will guide circuit assembly and firmware development in the tasks that follow.

The specification demonstrates that the selected components operate within a single 5V logic domain, communicate over a minimal two-wire I²C bus, and fit comfortably within the Arduino Uno's flash, SRAM, and EEPROM budget, while delivering an expected end-to-end response time well within the range required for real-time driver feedback. This design is well suited to

its purpose as an educational embedded systems prototype: it uses low-cost, readily available components, requires no custom PCB or specialised tooling, and demonstrates a complete sense-compute-actuate control loop suitable for direct internship submission and evaluation.